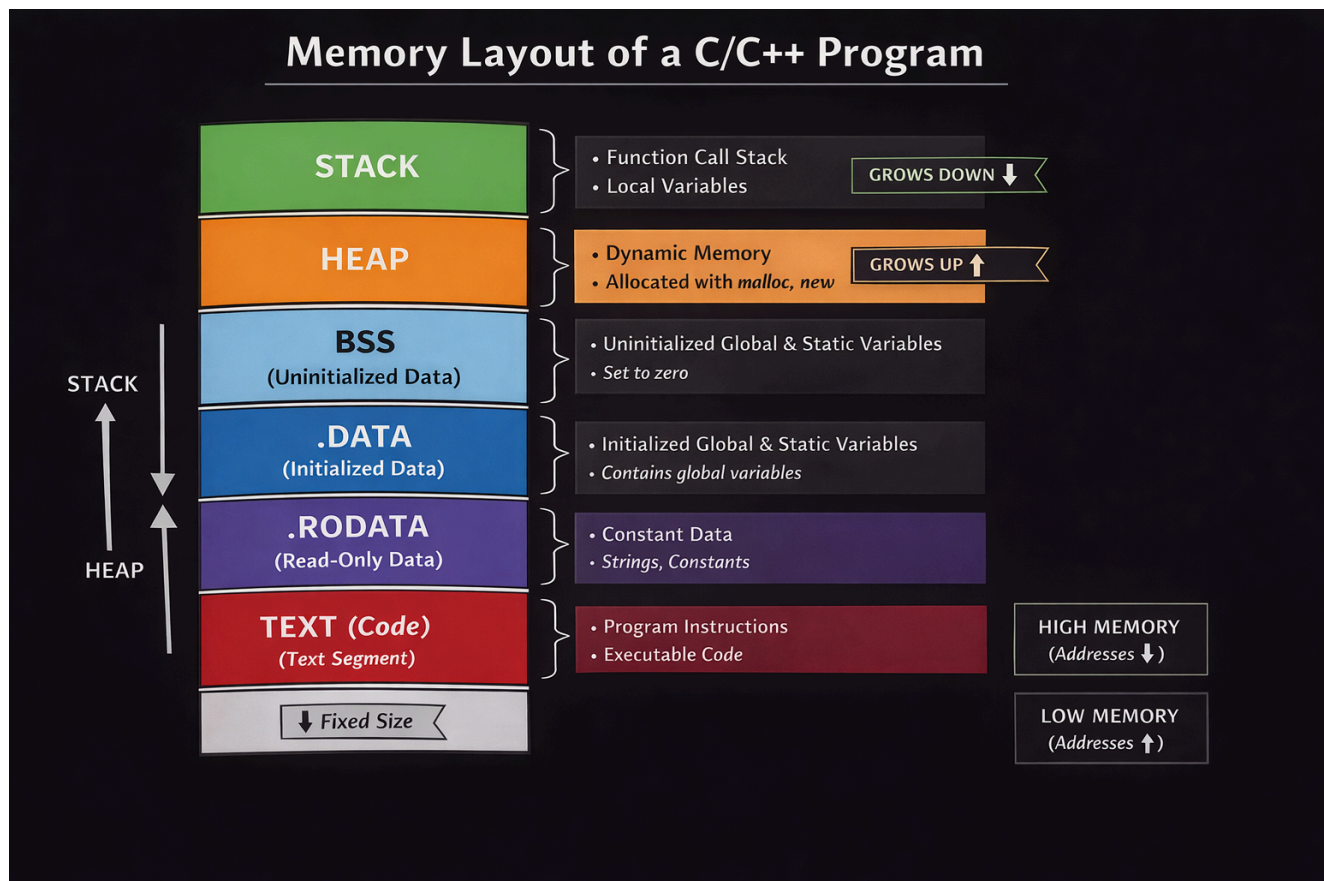


8. Memory Layout



Memory Layout of a C/C++ Program (Brief Explanation)

1. TEXT Segment (Code)

- Contains **compiled program instructions**
- This is your actual code (`main()` , functions, etc.)
- Usually **read-only**

👉 Example:

```
1 int main() { return 0; }
```

2. RODATA (Read-Only Data)

- Stores **constants and string literals**
- Cannot be modified

👉 Example:

```
1 printf("Hello"); // "Hello" stored here
```

3. DATA Segment (Initialized Data)

- Stores **global and static variables that are initialized**

👉 Example:

```
1 int x = 10; // goes here
```

4. BSS Segment (Uninitialized Data)

- Stores **global and static variables that are NOT initialized**
- Automatically initialized to 0

👉 Example:

```
1 int x; // goes here (value = 0 by default)
```

5. HEAP

- Used for **dynamic memory allocation**
- Managed manually (`malloc` , `new` , `free` , `delete`)
- **Grows upward**

👉 Example:

```
1 int* p = new int;
```

6. STACK

- Stores:
 - Function calls
 - Local variables
 - Parameters
- Managed automatically
- **Grows downward**

👉 Example:

```
1 void func() {  
2     int x = 5; // stored in stack  
3 }
```

⚠ Important Behavior

- **Stack grows downward** ↓
- **Heap grows upward** ↑
- If they collide → ❌ crash (stack overflow / heap overflow)

🧠 Mental Model

```
1 [ STACK ] ↓ (function calls, locals)  
2 [ HEAP  ] ↑ (dynamic memory)  
3 [ BSS   ] (uninitialized globals)  
4 [ DATA ] (initialized globals)  
5 [ RODATA ] (constants)  
6 [ TEXT  ] (code)
```

Key Takeaways

- Local variables → **Stack**
- Global/static → **Data / BSS**
- Dynamic memory → **Heap**

- Code → **Text segment**